Single Digit Handwritten Number Recognition

Introduction:

This is a personal project aiming to build a simple neural network that can recognize handwritten digits from 0-9 on a 28x28 pixels screen, nothing special.

Tools used:

Python3 for the codes,
NumPy for matrix and vector operations,
Tkinter for interactive GUI,
Plotly for result graphs,
MNIST dataset for training (60000 images) and testing (10000 images).

The Neural Network and the Maths:

This neural network consists of 3 layers:

1. The input layer with 784 neurons
2. The hidden layer with 128 neurons
3. The output layer with 10 neurons

Each neuron in the next layer is a weighted sum of all the neurons in the current layer plus some bias, and through an activation function.

Let $\mathbf{x} \in \mathbb{R}^{784}, \boldsymbol{y} \in \mathbb{R}^{128}, \mathbf{z} \in \mathbb{R}^{10}$ be the three layers of neurons represented as vectors. Let $\mathbf{b} \in \mathbb{R}^{128}, \mathbf{c} \in \mathbb{R}^{10}$ be the biases.

In this project, the sigmoid function:

$$\sigma: \mathbb{R} \to (0,1), \qquad \sigma(t) = \frac{1}{1+e^{-t}},$$

is used as the activation function.

Let

$$\mathbf{W} = \begin{pmatrix} w_{1\,1} & w_{1\,2} & \cdots & w_{1\,784} \\ w_{2\,1} & w_{2\,2} & \cdots & w_{2\,784} \\ \vdots & \vdots & \ddots & \vdots \\ w_{128\,1} & w_{128\,2} & \cdots & w_{128\,784} \end{pmatrix},$$

be the weights between **x** and **y**, and let

$$\mathbf{H} = \begin{pmatrix} h_{1\,1} & h_{1\,2} & \cdots & h_{1\,128} \\ h_{2\,1} & h_{2\,2} & \cdots & h_{2\,128} \\ \vdots & \vdots & \ddots & \vdots \\ h_{10\,1} & h_{10\,2} & \cdots & h_{10\,128} \end{pmatrix},$$

be the weights between **y** and **z**.

Let $\mathbf{Wx} + \mathbf{b} = \mathbf{u}$:

$$\begin{pmatrix} w_{1\,1} & w_{1\,2} & \cdots & w_{1\,784} \\ w_{2\,1} & w_{2\,2} & \cdots & w_{2\,784} \\ \vdots & \vdots & \ddots & \vdots \\ w_{128\,1} & w_{128\,2} & \cdots & w_{128\,784} \end{pmatrix} \begin{pmatrix} \mathrm{x}_1 \\ x_2 \\ \vdots \\ x_{784} \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_{128} \end{pmatrix} = \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{128} \end{pmatrix},$$

then $\mathbf{y} = \sigma(\mathbf{u})$, or formally:

$$\forall i \in \{1, \ldots, 128\},\ y_i = \sigma(u_i).$$

Similarly let $\mathbf{Hy} + \mathbf{c} = \mathbf{v}$:

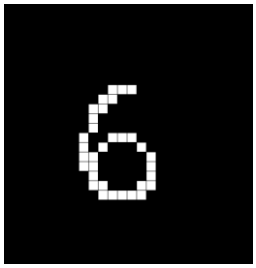
$$\begin{pmatrix} h_{1\,1} & h_{1\,2} & \cdots & h_{1\,128} \\ h_{2\,1} & h_{2\,2} & \cdots & h_{2\,128} \\ \vdots & \vdots & \ddots & \vdots \\ h_{10\,1} & h_{10\,2} & \cdots & h_{10\,128} \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_{128} \end{pmatrix} + \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_{10} \end{pmatrix} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_{10} \end{pmatrix},$$

then $\mathbf{z} = \sigma(\mathbf{v})$.

Define the cost function

$$C\colon \mathbb{R}^{10} \to \mathbb{R}, \qquad C(\mathbf{z}) = \frac{1}{2}(\mathbf{z} - \mathbf{a})^2, \qquad \mathbf{a} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_{10} \end{pmatrix},$$

where **a** is the vector represents the correct digit for the given input **x**, such that $a_i = 1$ for the correct digit i, and 0 for all other digits. For example, if this input was given:



then $a_6 = 1$, and $a_i = 0$ for all other $i \in \{0, 1, \dots, 9\} \backslash \{6\}$.

The purpose of this neural network is to minimise the cost function for any given input, ideally:

$$\begin{aligned} &\text{if} \quad a_i = 1 \quad \text{then} \quad z_i = 1, \text{and} \\ &\text{if} \quad a_i = 0 \quad \text{then} \quad z_i = 0. \end{aligned}$$

If $z_i = 1$, we say that the neural network 'believes' the given digit is i. But since z_i is a random number between 0-1, in this project the index of the maximum in $\mathbf{z}$ is taken as neural network's predicted answer.

How to minimise the cost function:

To minimise the cost function, we use the gradient descent method, in simple terms that is to calculate the gradient for each given input and take a step to the opposite direction of the gradient. We let the step to be proportional to the gradient, so the step gets smaller when it walks towards the minimum.

We want to manipulate the weight matrices and bias vectors, so that for each given input $\mathbf{x}$, we can output the $\mathbf{z}$ that makes the cost function as small as possible. We want these gradients:

$$\frac{\partial C}{\partial \mathbf{W}}, \frac{\partial C}{\partial \mathbf{H}}, \frac{\partial C}{\partial \mathbf{b}}, \frac{\partial C}{\partial \mathbf{c}}.$$

I haven't learnt partial derivatives with matrices and vectors, therefore please forgive me for what's about to happen in the next several pages.

We will find $\frac{\partial C}{\partial \mathbf{H}}$ and $\frac{\partial C}{\partial \mathbf{c}}$ first:

Consider we make some changes to $h_{\mathrm{i\,j}}$, then it will eventually impact C through the following chain:

$$h_{\mathrm{i\,j}} \to v_{\mathrm{i}} \to z_i \to C,$$

using the chain rule of partial derivatives:

$$\frac{\partial C}{\partial h_{i\,j}} = \frac{\partial C}{\partial z_i}\frac{\partial z_i}{\partial v_i}\frac{\partial v_i}{\partial h_{i\,j}}.$$

Since $C(z_i) = \frac{1}{2}(z_i - a_i)^2$,

$$\frac{\partial C}{\partial z_i} = (z_i - a_i).$$

Since $z_i = \sigma(v_i)$,

$$\frac{\partial z_i}{\partial v_i} = \sigma'(v_i).$$

Now since $v_i = \mathbf{H}_i\mathbf{y} + c_i$,
if $h_{i\,j}$ changes, then the rate which v_i changes is the coefficient of $h_{i\,j}$, which is y_j, therefore

$$\frac{\partial v_i}{\partial h_{i\,j}} = y_j.$$

Hence

$$\frac{\partial C}{\partial h_{i\,j}} = (z_i - a_i)\sigma'(v_i)y_j.$$

Similarly, if c_i changes, then it will impact C through the chain:

$$c_{\mathrm{i}} \to v_{\mathrm{i}} \to z_i \to C,$$

So

$$\frac{\partial C}{\partial c_i} = \frac{\partial C}{\partial z_i}\frac{\partial z_i}{\partial v_i}\frac{\partial v_i}{\partial c_i}.$$

The first two partial derivatives are the same as above. Now since $v_i = h_i\mathbf{y} + c_i$, the coefficient in front of c_i is 1, therefore

$$\frac{\partial v_i}{\partial c_i} = 1.$$

We have

$$\frac{\partial C}{\partial c_i} = (z_i - a_i)\sigma'(v_i).$$

Let $\boldsymbol{\delta}^{(2)} \in \mathbb{R}^{10}$ such that $\delta_i^{(2)} = \frac{\partial C}{\partial z_i}\frac{\partial z_i}{\partial v_i} = (z_i - a_i)\sigma'(v_i)$, then

$$\boxed{\frac{\partial C}{\partial h_{i\,j}} = y_j\delta_i^{(2)}, \text{ and } \frac{\partial C}{\partial c_i} = \delta_i^{(2)}.}$$

Now we find $\frac{\partial C}{\partial \mathbf{W}}$ and $\frac{\partial C}{\partial \mathbf{b}}$.

Same as above, if we change $w_{i\,j}$, then it will impact C through:

$$w_{i\,j} \to u_i \to y_i \to \mathbf{v} \to \mathbf{z} \to C.$$

Note that change in y_i will result in changes to all 10 entries in $\mathbf{v}$ and $\mathbf{z}$.

Using the chain rule:

$$\frac{\partial C}{\partial w_{i\,j}} = \frac{\partial C}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{v}} \frac{\partial \mathbf{v}}{\partial y_i} \frac{\partial y_i}{\partial u_i} \frac{\partial u_i}{\partial w_{i\,j}}.$$

By the total derivative rule, we must add up all the partial derivative in the 10 different chains from y_i to C, so

$$\begin{aligned} \frac{\partial C}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{v}} \frac{\partial \mathbf{v}}{\partial y_i} = \frac{\partial C}{\partial y_i} &= \sum_{k=1}^{10} \frac{\partial C}{\partial z_k} \frac{\partial z_k}{\partial v_k} \frac{\partial v_k}{\partial y_i} \\ &= \sum_{k=1}^{10} \delta_k^{(2)} \frac{\partial v_k}{\partial y_i} \\ &= \sum_{k=1}^{10} \delta_k^{(2)} h_{k\,i}, \end{aligned}$$

since $v_k = \mathbf{H}_k \mathbf{y} + c_k$, the coefficient of y_i is $h_{k\,i}$.

Therefore

$$\frac{\partial C}{\partial w_{i\,j}} = \frac{\partial y_i}{\partial u_i} \frac{\partial u_i}{\partial w_{i\,j}} \sum_{k=1}^{10} \delta_k^{(2)} h_{k\,i}.$$

Since $y_i = \sigma(u_i)$,

$$\frac{\partial y_i}{\partial u_i} = \sigma'(u_i).$$

Since $u_i = \mathbf{W}_i \mathbf{x} + b_i$, if $w_{i\,j}$ changes, then the rate which u_i changes is x_j, therefore

$$\frac{\partial u_i}{\partial w_{i\,j}} = x_j.$$

Hence

$$\frac{\partial C}{\partial w_{i\,j}} = x_j\, \sigma'(u_i) \sum_{k=1}^{10} \delta_k^{(2)} h_{k\,i}$$

Similarly, if we change b_i, this will impact C through:

$$b_i \to u_i \to y_i \to \mathbf{v} \to \mathbf{z} \to C,$$

therefore

$$\begin{aligned}\frac{\partial C}{\partial b_i} &= \frac{\partial C}{\partial \mathbf{z}}\frac{\partial \mathbf{z}}{\partial \mathbf{v}}\frac{\partial \mathbf{v}}{\partial y_i}\frac{\partial y_i}{\partial u_i}\frac{\partial u_i}{\partial b_i} \\ &= \frac{\partial u_i}{\partial b_i}\,\sigma'(u_i)\sum_{k=1}^{10}\delta_k^{(2)}h_{k\,i} \\ &= \sigma'(u_i)\sum_{k=1}^{10}\delta_k^{(2)}h_{k\,i},\end{aligned}$$

since $u_i = \mathbf{W}_i\mathbf{x} + b_i$, the coefficient of b_i is 1.

We let $\boldsymbol{\delta}^{(1)} \in \mathbb{R}^{128}$ such that $\delta_i^{(1)} = \sigma'(u_i)\sum_{k=1}^{10}\delta_k^{(2)}h_{k\,i}$, then

$$\boxed{\frac{\partial C}{\partial w_{i\,j}} = x_j\delta_i^{(1)},\ \text{and}\ \ \frac{\partial C}{\partial b_i} = \delta_i^{(1)}.}$$

We have found the gradients!

Now we turn them back into matrices and vectors so they look better:

Since

$$\frac{\partial C}{\partial c_i} = \delta_i^{(2)},\ \text{and}\ \ \frac{\partial C}{\partial b_i} = \delta_i^{(1)},$$

we have

$$\boxed{\frac{\partial C}{\partial \mathbf{c}} = \boldsymbol{\delta}^{(2)},\ \text{and}\ \ \frac{\partial C}{\partial \mathbf{b}} = \boldsymbol{\delta}^{(1)}.}$$

Since

$$\frac{\partial C}{\partial h_{i\,j}} = y_j\delta_i^{(2)},\ \text{and}\ \ \frac{\partial C}{\partial w_{i\,j}} = x_j\delta_i^{(1)},$$

so

$$\boxed{\frac{\partial C}{\partial \mathbf{H}} = \boldsymbol{\delta}^{(2)}\mathbf{y}^T,\ \text{and}\ \ \frac{\partial C}{\partial \mathbf{W}} = \boldsymbol{\delta}^{(1)}\boldsymbol{x}^{\mathrm{T}}}$$

So for each given input **x,** we compute the corresponding result **z**. Then we find the gradients introduced above, and we adjust the weights and biases using the gradients:

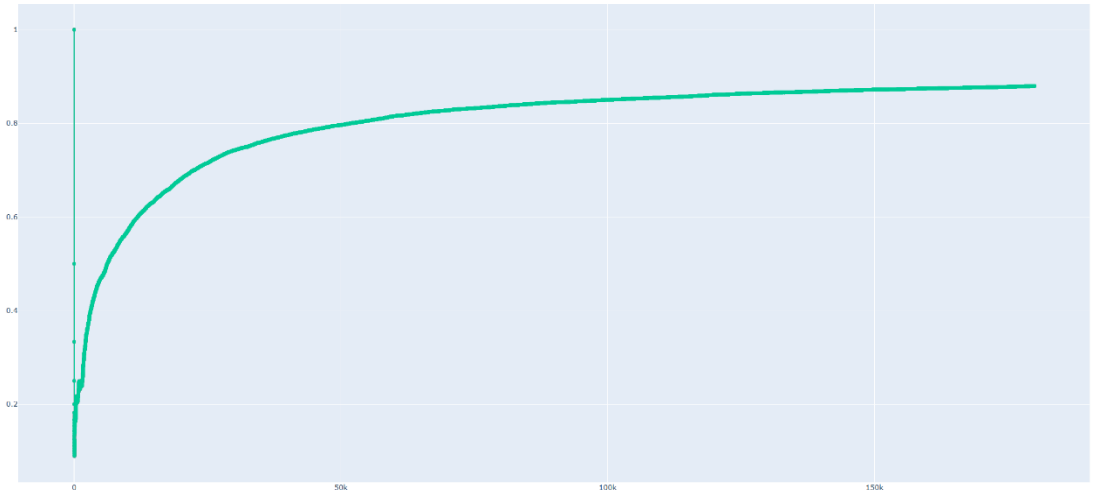
$$w_{new} = w_{old} - n\frac{\partial C}{\partial w},$$

where n is called the learning rate, is set to 0.01 in this project. Through multiple rounds of adjustment i.e. training, we will walk towards some local minimum of the cost function, and the neural network should somewhat be able to recognize handwritten digits.

Results:

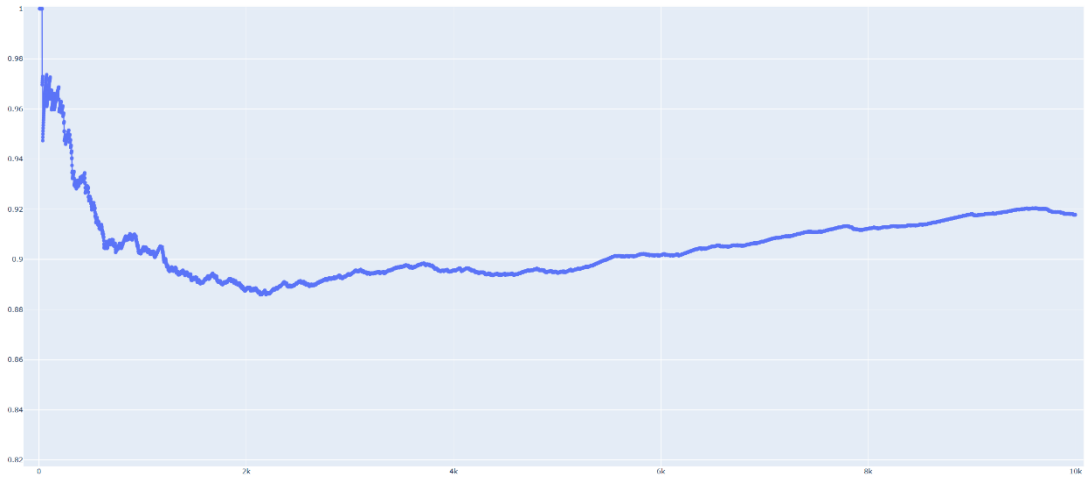
The training process was done by using the 60,000 labelled images from MNIST dataset, each pixel is processed before training such that the value is either 0 or 1 to match my Tkinter code, not 0-255 as in the original image.

In the training set:



The horizontal axis represents the number of training processes. The vertical axis represents the success rate. After 3 rounds of repeated training on the same dataset which contains 60,000 images, the success rate kept increasing and reaches 90% at the end.

In the testing set:



After 180k rounds of training, the neural network is tested by new 10,000 images which it hasn't seen before. The neural network is not trained in this run. The success rate drops from almost 98% at the first 2,000 rounds, and ends at about 90%, similar in the training dataset.

However, when I test the neural network manually on, the success rate was lot lower than 90%. I tried a few rounds and the result was catastrophic. Maybe it's because digits are centre placed while my handwritten digits are not.

:(

Reference:

3blue1brown Neural Networks series:
https://www.youtube.com/watch?v=aircAruvnKk&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi